

MSA 실습 서브노트

이름: 윤소영
담당 역할: #2 통합 분양공고 (course-service)
프로젝트명: HomeOne — B2B2C 분양 정보 통합 플랫폼

1. 고객 Pain Point & 서비스 제안

문제 인식

현재 국내 청약 시장은 크게 두 개의 플랫폼으로 나뉘어져 있습니다.

- 청약홈 (한국부동산원): 민영·공공 분양 공고
- LH청약플러스 (한국토지주택공사): LH 공공 임대·분양 공고

청약 신청자는 두 플랫폼을 각각 방문해야 하고, 원하는 조건(지역, 예산, 공급 유형)으로 통합 검색이 불가능합니다. 또한 인기 단지의 청약 접수 기간에는 동시 접속자가 폭증하여 플랫폼 전체가 느려지거나 장애가 발생하는 문제가 있습니다.

서비스 제안: HomeOne

"여러 공급사의 분양 정보를 한 곳에서 조회하고, 조건에 맞는 단지를 추천받으며, 청약 신청까지 완결하는 B2B2C 통합 플랫폼"

층	대상	제공 기능
B (공급)	시행사·건설사	분양 공고 등록, 접수 현황 확인
플랫폼	HomeOne	데이터 통합, 추천, 청약 처리
C (수요)	청약 신청자	통합 조회, 맞춤 추천, 청약 신청

2. 아키텍처 명세

왜 MSA(마이크로서비스 아키텍처)를 선택했는가

기존 모놀리식 구조에서는 청약 접수 서버에 부하가 집중되면 분양 정보 조회 서비스도 함께 중단됩니다. MSA는 서비스를 독립된 단위로 분리하여, 한 서비스의 장애가 다른 서비스에 영향을 주지 않습니다.

모놀리식: 청약 서버 과부하 → 전체 서비스 중단

MSA: 청약 서버 과부하 → 분양 조회는 정상 작동 (서비스 격리)

서비스 구성

서비스	포트	역할
user-service	8081	회원가입, 로그인
course-service	8082	분양 공고 조회·등록 (담당)
enrollment-service	8083	청약 접수·상태 관리
payment-service	8084	수수료 결제 처리
recommend-service	8085	AI 기반 맞춤 단지 추천
eureka-server	8761	서비스 디스커버리
api-gateway	8080	단일 진입점, 인증 처리

서비스 간 통신 방식

- 동기 (REST): 즉시 응답이 필요한 경우 — 예) 청약 접수 시 단지 존재 여부 확인
- 비동기 (Kafka): 느슨한 연결이 필요한 경우 — 예) 결제 완료 → 청약 상태 자동 변경

결제 서버가 Kafka에 "결제 완료" 메시지 발행

→ 청약 서버가 메시지 수신

→ 청약 상태를 PENDING → ACTIVE 로 자동 변경

 캡처 권장: Eureka 대시보드 (<http://localhost:8761>) — 모든 서비스가 UP 상태인 화면

3. Agile 방법론 적용

Sprint 구성

Sprint 1 목표: 서비스 기동 확인 및 시드 데이터 구성, 기본 흐름 검증

담당	Sprint 1 작업
#1	(팀원 작업 내용)
#2 (나)	공급사 DB 등록, 분양 단지 시드 데이터 적재, 공공 API 연동
#3	(팀원 작업 내용)
#4	(팀원 작업 내용)

Sprint 2 목표: 공공 API 실시간 연동, 화면 연결 완성, 발표 준비

User Story (내 담당)

우선순위	내용	완료 기준
● 최고	공급사 5개 users 테이블에 등록	INSERT 성공, FK 에러 없음
● 최고	분양 단지 시드 데이터 적재	GET /api/courses → 200 + 데이터
중	공공 API 연동 스크립트 작성	청약홈·LH 데이터 정규화 성공
중	접수 건수 편차 부여	추천 서비스 정렬 정상 작동

4. API 개발 내용 (담당 파트)

역할 한 줄 요약

"청약홈(한국부동산원)과 LH(한국토지주택공사) 두 공공 API에서 분양 데이터를 수집하여, 단일 형태로 정규화한 뒤 courses 테이블에 적재합니다."

DB 설계 — courses 테이블 컬럼 재해석

교수님이 제공한 01_init.sql 의 courses 테이블 구조를 그대로 활용하되, 컬럼을 분양 도메인에 맞게 재해석하였습니다.

원본 컬럼명	분양 도메인 의미	예시 값
title	단지명 + 주택형	더샵 신길센트럴시티 59A

category	공급 지역 (enum 재해석)	SECURITY = 서울
price	분양가 (만원 단위)	124000 = 12.4억
instructor_id	공급사 ID	103 = 포스코이앤씨
enrollment_count	청약 접수 건수	2870
status	접수 상태	ACTIVE = 접수중
description	추가 정보 (JSON)	주소, 신청 기간, 상세 URL 등

공급사 시드 데이터 (02_seed_suppliers.sql)

courses 테이블의 `instructor_id` 가 users 테이블을 외래키로 참조하기 때문에, 분양 단지 데이터보다 공급사를 먼저 등록해야 합니다. id를 101번부터 시작한 이유는 기존 DB에 id 1(일반 회원), id 2(강사 계정)가 이미 존재하여 충돌을 방지하기 위함입니다.

```
INSERT INTO users (id, name, role, ...) VALUES
  (101, '한국토지주택공사', 'INSTRUCTOR', ...),
  (102, '경기주택도시공사', 'INSTRUCTOR', ...),
  (103, '포스코이앤씨', 'INSTRUCTOR', ...),
  (104, '지에스건설', 'INSTRUCTOR', ...),
  (105, '대우건설', 'INSTRUCTOR', ...);
```

분양 단지 시드 데이터 (03_seed_courses.sql)

청약홈 공공 API에서 수집한 실제 분양 공고 데이터를 적재합니다. 하나의 공고에 여러 주택형이 존재하므로, 주택형별로 courses 행을 각각 생성합니다.

- 시티오씨엘 9단지 오션파크뷰 59 / 75 / 84A / 84B / 95 ... (9개 행)
- 더샵 신길센트럴시티 51A / 51B / 51C ... (10개 이상)
- 성남복정2 신혼희망타운 55A / 55B / 55C ...

공공 API 연동 (import_notices.py)

두 기관의 API 응답 형태가 다르기 때문에, 하나의 통합 모델로 정규화하는 작업이 필요했습니다.

항목	청약홈 API	LH API
응답 형태	{ "data": [...] }	[{dsSch}, {dsList}] 2원소 배열

분양가 필드	LTTOT_TOP_AMOUNT (만원 단위)	없음 (목록 API 미제공)
지역 필드	SUBSCRIPT_AREA_CODE_NM	CNP_CD_NM

역등성 보장: title 기준으로 중복 체크하여 동일 스크립트를 여러 번 실행해도 데이터가 중복 생성되지 않습니다.

 캡처 권장: DBeaver에서 courses 테이블 데이터 확인 화면

5. API 명세서

course-service의 주요 API 엔드포인트입니다. (Swagger UI: <http://localhost:8082/swagger-ui.html>)

메서드	경로	설명
GET	/api/courses	분양 단지 전체 목록 조회
GET	/api/courses/{id}	단지 상세 정보 조회
POST	/api/courses	단지 등록 (공급사 계정 필요)
GET	/api/courses/search	카테고리·키워드 검색
POST	/api/courses/internal/import	공공 API 데이터 적재 트리거

 캡처 권장 (필수)

1. Swagger UI 화면 전체 (API 목록)
2. GET /api/courses → Execute → 200 OK + 데이터 응답
3. POST /api/courses → Execute → 201 Created

6. 서비스 동작 확인

 캡처 권장: <http://localhost:8761> — 전체 서비스 UP 상태

 캡처 권장: <http://localhost:3000> — 분양 단지 목록 화면

7. 배운 점 & 느낀 점

MSA 구조에 대한 이해

처음에는 서버가 왜 여러 개인지, 하나로 만들면 안 되는지 이해하지 못했습니다. 실습을 통해 enrollment-service를 종료해도 course-service가 정상 동작하는 것을 확인하면서, MSA의 핵심인 **서비스 격리**를 체감할 수 있었습니다.

Docker와 컨테이너의 역할

Docker라는 단어는 알고 있었지만, 컨테이너·이미지·docker-compose의 차이가 무엇인지 전혀 몰랐습니다. 실습을 통해 Dockerfile은 레시피, 이미지는 밀키트, 컨테이너는 실제 조리된 음식이라는 관계를 이해하게 되었습니다. docker-compose.yml 파일 하나로 10개의 서비스가 순서에 맞춰 자동으로 기동되는 과정이 인상적이었습니다.

API의 의미

API라는 단어를 처음 접했을 때는 "API 키"라는 개념으로만 알고 있어서, API를 어떤 열쇠나 인증 수단으로 오해하고 있었습니다. 실습을 통해 API는 두 프로그램이 대화하는 창구이며, API 키는 그 창구를 사용할 수 있는 허가증임을 이해했습니다.

DB와 데이터 흐름

테이블 생성(DDL)과 데이터 삽입(DML)이 다른 작업임을 몰랐고, 외래키(Foreign Key)로 인해 users 테이블에 공급사 데이터가 없으면 courses 테이블에 데이터를 넣을 수 없다는 것도 직접 에러를 마주하며 배웠습니다.

공공 API 연동의 어려움

청약홈과 LH 두 기관의 API 응답 형태가 완전히 달랐습니다. 두 소스를 하나의 통일된 모델로 정규화하는 작업이 실제 데이터 엔지니어링에서 겪는 어려움을 미리 경험하게 해주었습니다.

Agile 방법론 적용 소감

역할을 나눠서 동시에 작업하다 보니, 내 작업이 팀 전체의 병목이 될 수 있다는 것을 처음으로 체감했습니다. courses 테이블에 데이터가 없으면 청약 서버도, AI 추천 서버도 의미 있는 테스트를 할 수 없었기 때문입니다.

캡처 체크리스트 (내일 확인)

- [] localhost:8761 — Eureka 대시보드, 전체 서비스 UP
- [] localhost:8082/swagger-ui.html — Swagger UI API 목록
- [] Swagger GET /api/courses → 200 OK + 데이터
- [] Swagger POST /api/courses → 201 Created
- [] DBeaver courses 테이블 데이터 뷰
- [] localhost:3000 — Vue 프론트엔드 분양 목록